

Quy hoạch động và tính đơn điệu

Trương Kiếm Thính Vũ

Nguồn gốc: Dynamic Programming and Monotonicity

Dynamic Programming / DP and Monotonicity.pdf

Lời nói đầu

Quy hoạch động và tính đơn điệu là một cặp không thể tách rời. Nhiều bài toán quy hoạch động, do dạng đặc biệt của phương trình chuyển trạng thái, có tính đơn điệu ở quyết định hoặc ở giá trị. Những tính chất đặc biệt này thường trở thành điểm đột phá quan trọng. Xu hướng ra đề hiện nay ngày càng đa dạng và mở rộng; quy hoạch động thuần túy trong nhiều trường hợp bị giới hạn bởi nhiều yếu tố. Vì vậy, việc phát hiện và ứng dụng tính đơn điệu dần trở thành một chủ đề quan trọng. Dưới đây, ta thông qua một số ví dụ để nắm bắt tư tưởng này.

Do tối ưu quyết định bằng hàng đợi và hệ số góc đã được xếp vào phần cấu trúc dữ liệu, bài này không trình bày lại.

1 Ví dụ 1: Bài toán đặt cơ sở dịch vụ

Đề bài

Trong một thành phố được chia thành các khu phố đều đặn theo hướng nam–bắc, n điểm dân cư nằm trên n tọa độ thẳng hàng

$$x_1 < x_2 < x_3 < \dots.$$

Người dân muốn chọn ít nhất 1 điểm dân cư, nhưng không quá k điểm dân cư, để xây dựng cơ sở dịch vụ.

Tại điểm dân cư i , nhu cầu dịch vụ là $w_i \geq 0$, và chi phí xây cơ sở dịch vụ tại điểm này là $c_i \geq 0$. Giả sử khoảng cách từ điểm dân cư i đến cơ sở dịch vụ gần nhất là d_i , khi đó chi phí dịch vụ của điểm dân cư này là $d_i w_i$.

Tổng chi phí khi xây k cơ sở dịch vụ là $A + B$. Ở đây A là tổng chi phí xây cơ sở dịch vụ tại k điểm dân cư được chọn, còn B là tổng chi phí dịch vụ của n điểm dân cư.

Nhiệm vụ là: với n điểm cho trước trên đường thẳng L , hãy lập trình tính tổng chi phí nhỏ nhất khi đặt nhiều nhất k cơ sở dịch vụ trên L .

Ghi chú: $N \leq 1000$.

Phân tích

Bài toán này gợi nhớ đến một mô hình quy hoạch động kinh điển: bài toán bưu điện. Ở đây chỉ là phiên bản có trọng số. Vì vậy, bất cứ bài toán bưu điện ta rất dễ nhận được phương trình chuyển trạng thái sau.

Gọi $f[i, j]$ là chi phí nhỏ nhất khi đặt j cơ sở dịch vụ trên các điểm dân cư $1, \dots, i$, và cơ sở dịch vụ thứ j đặt tại điểm dân cư i . Khi đó

$$f[i, j] = \min_{1 \leq k \leq i-1} \{f[k, j-1] + \text{value}(k+1, i)\} + c_i.$$

Trong đó, $\text{value}(k+1, i)$ là chi phí dịch vụ nhỏ nhất của các điểm dân cư từ $k+1$ đến i khi hai cơ sở dịch vụ trên hai điểm k và i là hai cơ sở kề nhau. Giá trị này có thể được tính trước trong $O(n^2)$.

Phương trình trên có độ phức tạp $O(n^3)$, hiển nhiên sẽ quá thời gian. Ta cần tìm một số cách tối ưu.

Gọi $s[i, j]$ là giá trị k tại đó $f[i, j]$ đạt giá trị nhỏ nhất, tức quyết định của $f[i, j]$. Ta quan sát tính đơn điệu của $s[i, j]$:

$$\text{I. } s[i, j] < s[i+1, j].$$

Giải thích trực giác: khi ta muốn phân bố “đều” j cơ sở dịch vụ cho một số điểm dân cư, số điểm dân cư cần xử lý càng nhiều thì vị trí cơ sở dịch vụ càng lùi về sau.

$$\text{II. } s[i, j] > s[i, j-1].$$

Giải thích trực giác: khi ta muốn phân bố “đều” một số cơ sở dịch vụ cho i điểm dân cư, số cơ sở dịch vụ cần xử lý càng nhiều thì vị trí của chúng càng lùi về sau.

Hai kết luận trên đều có thể được chứng minh chặt chẽ bằng toán học; ở đây ta lược bỏ. Trong phòng thi, ta cũng có thể in mảng s thành một ma trận vuông, từ đó dễ dàng quan sát quy luật và thu được các kết luận trên.

Gộp I và II, ta nhận được một công thức rất quan trọng:

$$s[i, j-1] < s[i, j] < s[i+1, j].$$

Công thức này cho ta biết: mỗi quyết định đều có một phạm vi, không cần quét từ đầu đến cuối. Vấn đề hiện tại là làm thế nào biết được phạm vi quyết định trước khi tính một $f[i, j]$ nào đó.

Thật ra cách làm rất đơn giản: điều chỉnh thứ tự tính toán. Ở vòng lặp ngoài, ta tính j từ nhỏ đến lớn; ở vòng lặp trong, ta tính i từ lớn đến nhỏ. Khi đó, trước khi $f[i, j]$ được tính ra, ta có thể bảo đảm rằng $s[i, j-1]$ và $s[i+1, j]$ đều đã biết.

Nhìn trên chương trình, tuy bề ngoài vẫn là ba vòng lặp, hiệu quả chạy thực tế rất tốt. Độ phức tạp thời gian xấp xỉ nằm trong khoảng $O(n^2)$ đến $O(n^2 \log n)$. Bài toán này nhờ đó được giải quyết tốt.

2 Ví dụ 2: Ngôi nhà nhỏ của H

Đề bài

H thề sẽ trở thành nhà toán học vĩ đại nhất thế kỷ 21. Cậu cho rằng làm nhà toán học cũng giống như làm ca sĩ: bước đầu tiên là phải đóng gói hình ảnh cho tốt, nếu không thì dù tài năng đến đâu cũng không thể quảng bá được. Vì vậy, cậu quyết định trước hết đầu tư vào nơi ở của mình, để người khác nhìn vào là biết bên trong có một “nhà toán học lớn trong tương lai”.

Để tiện mô tả, ta lấy hướng đông làm chiều dương của trục x , hướng bắc làm chiều dương của trục y , và thiết lập hệ tọa độ Descartes trên mặt phẳng. Nhà của H có chiều đông–tây dài 100 Hil; Hil là đơn vị độ dài do H tự dùng, còn đổi ra mét thế nào thì không ai biết. Tường đông và tường tây đều song song với trục y ; tường bắc và tường nam lần lượt là hai đường thẳng có hệ số góc k_1 và k_2 , trong đó k_1, k_2 là các số thực dương. Ở góc tường bắc và góc tường nam có nhiều thảm cỏ, mỗi thảm cỏ là một hình chữ nhật, và mỗi cạnh của hình chữ nhật đều song song với các trục tọa độ. Điểm tiếp xúc của hai thảm cỏ kề

nhau nằm đúng trên tường; hoành độ của điểm tiếp xúc được gọi là một “điểm chia” trên bức tường đó, và các điểm chia này phải là các số nguyên từ 1 đến 99.

H cho rằng sự kết hợp giữa đối xứng và bất đối xứng mới thể hiện đầy đủ “vẻ đẹp toán học”. Vì vậy, ở góc tường bắc phải có m thăm cỏ, ở góc tường nam phải có n thăm cỏ, và quy ước $m \leq n$. Nếu ký hiệu tập điểm chia của tường bắc và tường nam lần lượt là X_1 và X_2 , thì phải có $X_1 \subseteq X_2$, nghĩa là mọi điểm chia trên tường bắc nhất định cũng là điểm chia trên tường nam.

Do H hiện chưa có thu nhập dồi dào, cậu phải hạ chi phí làm thăm cỏ xuống thấp nhất, tức làm cho tổng diện tích chiếm đất của các thăm cỏ nhỏ nhất. Cậu nhờ bạn giúp tính diện tích này.

Ghi chú: $2 \leq m \leq n \leq 100$. Khoảng cách giữa tường nam và tường bắc rất lớn, nên không xảy ra trường hợp thăm cỏ ở tường nam và tường bắc chồng lên nhau.

Phân tích

Có một điều kiện rất nổi bật: “mọi điểm chia trên tường bắc nhất định cũng là điểm chia trên tường nam”. Điều này có nghĩa là, sau khi xác định các điểm chia của tường bắc, ta có thể xác định cách chia của tường nam theo từng đoạn.

Gọi $f[w, u, v]$ là diện tích nhỏ nhất khi chiều dài thăm cỏ hiện xét trên hai tường nam–bắc là w , tường bắc có u thăm cỏ và tường nam có v thăm cỏ. Ta có

$$f[w, u, v] = \min\{f[w - x, u - 1, v - k] + \text{area}(x, k)\}.$$

Trong đó, x là độ dài của thăm cỏ thứ u trên tường bắc được liệt kê, k là số thăm cỏ trên tường nam tương ứng với thăm cỏ thứ u của tường bắc. Hàm area là giá trị nhỏ nhất của phần diện tích tăng thêm này và có thể được tính trong thời gian hằng số.

Đáp án là $f[100, m, n]$, độ phức tạp thời gian là $O(l^2 mn^2)$, chắc chắn quá thời gian.

Vậy cách làm này chậm ở đâu? Nguyên nhân là mỗi khi tính một trạng thái, ta phải liệt kê hai giá trị x và k , điều này rất không đáng. Ta thử giảm thời gian tiêu tốn ở phần này.

Giả sử ta đã xác định x , vậy k có quan hệ gì với $f[w, u, v]$? Nếu vẽ đồ thị của giá trị ứng viên theo sự thay đổi của k , ta sẽ ngạc nhiên phát hiện rằng đồ thị có dạng tương tự một parabol. Gọi p là điểm chuyển từ giảm sang tăng.

Từ đồ thị có thể thấy p chính là quyết định tối ưu. Ta tiếp tục khảo sát quan hệ giữa x và p , và phát hiện rằng khi x tăng, p sẽ không giảm.

Do cả x và p đều tăng, ta có thể vừa liệt kê x vừa dời vị trí của p về phía trước. Như vậy ta đã loại bỏ được việc liệt kê k .

Khi tính mỗi $f[w, u, v]$, ta chỉ liệt kê một giá trị x , nên độ phức tạp thời gian giảm xuống $O(l^2 mn)$, đủ để xử lý toàn bộ dữ liệu.

3 Ví dụ 3: Trứng đại bàng

Đề bài

Một giáo sư muốn xác định độ cứng E của một loại trứng đại bàng. Ông xác định E bằng cách liên tục thả trứng đại bàng từ một tòa nhà N tầng xuống. Nếu quả trứng thả từ tầng K không vỡ nhưng thả từ tầng $K + 1$ thì vỡ, khi đó $E = K$ với $0 \leq E \leq N$.

Nếu trong một lần thử quả trứng không vỡ, nó có thể tiếp tục được sử dụng. Nếu trong một lần thử quả trứng bị vỡ, giáo sư chỉ có thể lấy một quả trứng mới để tiếp tục thí nghiệm. Nếu dùng hết toàn bộ

trứng mà vẫn chưa xác định được E , ta xem như toàn bộ thí nghiệm thất bại. Hiện giáo sư có M quả trứng, ông muốn biết số lần thử ít nhất cần dùng để xác định E trong điều kiện thí nghiệm thành công.

Phân tích

Bài này yêu cầu tìm giá trị tối ưu, vì vậy ta nghĩ đến quy hoạch động.

Gọi $f[i, j]$ là số lần thử cần thiết trong trường hợp xấu nhất để xác định E khi dùng i quả trứng trên một tòa nhà j tầng.

Rõ ràng $f[1, j] = j$ với $j \geq 0$. Điều này có nghĩa là nếu ta chỉ có một quả trứng, để bảo đảm thí nghiệm thành công, ta chỉ có thể thử từng tầng từ dưới lên; trường hợp xấu nhất đương nhiên là j .

Khi $i \geq 2$, nếu ta thả một quả trứng từ tầng w :

1. Trứng vỡ. Khi đó $E < w$, nên ta chỉ có thể dùng $i - 1$ quả trứng để xác định E trong $w - 1$ tầng phía dưới. Đây là một bài toán con tương tự bài toán ban đầu, có đáp án $f[i - 1, w - 1]$. Khi đó

$$f[i, j] = f[i - 1, w - 1] + 1.$$

2. Trứng không vỡ. Khi đó $E \geq w$, nên ta sẽ xác định E trong $j - w$ tầng phía trên. Nếu trừu tượng hóa $j - w$ tầng này thành một tòa nhà mới, bài toán cũng tương tự bài toán ban đầu, có đáp án $f[i, j - w]$. Khi đó

$$f[i, j] = f[i, j - w] + 1.$$

Tổng hợp lại,

$$f[i, j] = \min_{1 \leq w \leq j} \{\max\{f[i - 1, w - 1], f[i, j - w]\}\} + 1.$$

Độ phức tạp là $O(mn^2) = O(n^3)$. Ta có thể thảo luận quan hệ lớn nhỏ giữa m và n để giảm xuống $O(n^2 \log n)$, nhưng vẫn cần thêm tối ưu.

Suy nghĩ tiếp: nếu ta có thể dùng i quả trứng để xác định một tòa nhà j tầng trong k lần, thì tương tự, dùng không quá k lần ta có thể xác định một tòa nhà $j - 1$ tầng, nhưng chưa chắc xác định được tòa nhà $j + 1$ tầng. Viết bằng ngôn ngữ toán học:

$$f[i, j] \geq f[i, j - 1].$$

Vì vậy, nếu $f[i - 1, w - 1] < f[i, j - w]$, thì với $v < w$ nhất định có $f[i, j - v] \geq f[i, j - w]$, do đó

$$\begin{aligned} \max\{f[i - 1, v - 1], f[i, j - v]\} + 1 &\geq f[i, j - v] + 1 \\ &\geq f[i, j - w] + 1 \\ &= \max\{f[i - 1, w - 1], f[i, j - w]\} + 1. \end{aligned}$$

Ta rút ra kết luận: v nhất định không phải quyết định tối ưu.

Tương tự, nếu $f[i - 1, w - 1] > f[i, j - w]$, thì với $v > w$, v nhất định không phải quyết định tối ưu.

Nếu đặt $f[i - 1, w - 1]$ và $f[i, j - w]$ trên cùng một trục tọa độ, ta có thể thấy đồ thị của giá trị lớn hơn trong hai giá trị này có dạng chữ “V”.

Từ hai điểm trên, ta có thể tìm nhị phân trên w : mỗi lần lấy một w ở giữa; nếu $f[i - 1, w - 1] > f[i, j - w]$ thì loại bỏ các quyết định lớn hơn w , ngược lại loại bỏ các quyết định nhỏ hơn w . Cuối cùng, quyết định còn lại nhất định là tối ưu.

Như vậy độ phức tạp cho mỗi quyết định giảm xuống $O(\log n)$, tổng độ phức tạp giảm xuống $O(n \log n \log n)$. Bước tối ưu đầu tiên đã thành công. Có thể tiếp tục cải thiện hay không?

Suy nghĩ tiếp: nếu $f[i, j]$ bằng k , thì trên cơ sở này, để xác định tòa nhà $j + 1$ tầng, ta cần nhiều nhất thêm 1 lần so với k , vì ta có thể trước hết thu được $f[i, j]$, rồi thử thêm một lần ở tầng $j + 1$. Viết bằng ngôn ngữ toán học:

$$f[i, j - 1] \leq f[i, j] \leq f[i, j - 1] + 1.$$

Vì vậy, nếu tồn tại một quyết định w làm cho $f[i, j] = f[i, j - 1]$, thì $f[i, j] = f[i, j - 1]$; ngược lại $f[i, j] = f[i, j - 1] + 1$.

Ta duy trì một p sao cho luôn thỏa

$$f[i, p] = f[i, j - 1] - 1, \quad f[i, p + 1] = f[i, j - 1].$$

Theo kết luận ở trên, một p như vậy nhất định tồn tại. Khi tính $f[i, j]$, ta đặt $w = j - p$ để tìm mối liên hệ bên trong.

$$\text{I. Nếu } f[i, p] \geq f[i - 1, j - p - 1],$$

thì

$$\begin{aligned} f[i, j] &= \max\{f[i - 1, w - 1], f[i, j - w]\} + 1 \\ &= \max\{f[i - 1, j - p - 1], f[i, p]\} + 1 \\ &= f[i, p] + 1 \\ &= f[i, j - 1] - 1 + 1 = f[i, j - 1]. \end{aligned}$$

Điều này cho thấy quyết định w hiện tại có thể làm cho $f[i, j] = f[i, j - 1]$.

$$\text{II. Nếu } f[i, p] < f[i - 1, j - p - 1],$$

lấy tùy ý một l . Nếu $l < p$, thì

$$\begin{aligned} f[i, j] &= \max\{f[i, l], f[i - 1, j - l - 1]\} + 1 \\ &\geq f[i - 1, j - l - 1] + 1 \\ &> f[i, p] + 1 = f[i, j - 1]. \end{aligned}$$

Suy ra $f[i, j] > f[i, j - 1]$, nên không thể có $f[i, j] = f[i, j - 1]$.

Tương tự, khi $l = p$ và khi $l > p$, quyết định w hiện tại đều không thể làm cho $f[i, j] = f[i, j - 1]$. Theo kết luận bên trên, khi đó $f[i, j] = f[i, j - 1] + 1$.

Tổng hợp lại, ta nhận được kết luận sau:

$$\text{khi } f[i, p] < f[i - 1, j - p - 1] \text{ thì } f[i, j] = f[i, j - 1] + 1,$$

ngược lại

$$f[i, j] = f[i, j - 1].$$

Với kết luận này, ta có thể chuyển trạng thái trong $O(1)$, và độ phức tạp thời gian được tối ưu thêm thành $O(n \log n)$. Đây đã là một thuật toán rất tốt.

Tổng kết

Từ ba bài toán trên có thể thấy, trong tư duy ta đều trải qua các bước: ý tưởng ban đầu, không khả thi, tìm tính đơn điệu để tối ưu, rồi giải quyết được bài toán. Ta cũng có thể thấy rằng tính đơn điệu trong tối ưu quy hoạch động là một lưỡi dao sắc, giúp ta vượt qua nhiều cửa ải. Nhưng nói đi cũng phải nói lại: nếu ta không nắm chắc các kỹ thuật này, không có nền tảng nhất định về suy luận, tư duy và toán học, thì lưỡi dao sắc ấy cũng chỉ có thể nằm trong vỏ mà thôi.